2024 18th International Conference on Advanced Computing and Analytics (ACOMPA) | 979-8-3315-4246-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/ACOMPA64883.2024.00019

Performance Evaluation of Decentralized Machine Learning based Network-Based Intrusion Detection System for Internet of Things

Duc Vu-Minh[1, 2], My Duong-Tran-Tra[1, 2], Luan Van-Thien[1, 2],
Anh Pham-Nguyen-Hai[1, 2], Thuat Nguyen-Khanh[1, 2, *], and Quan Le-Trung[1, 2]
[1]*Faculty of Computer Networks and Communications, University of Information Technology, Ho Chi Minh City, Viet Nam*
[2]*Vietnam National University, Ho Chi Minh City, Vietnam*
20520163@gm.uit.edu.vn, 20520640@gm.uit.edu.vn, luanvt@uit.edu.vn,
21520586@gm.uit.edu.vn, thuatnk@uit.edu.vn, quanlt@uit.edu.vn
[*]Corresponding Author

***Abstract*—In recent years, IoT applications have become increasingly popular. Smart services have been deployed from the IoT infrastructure to provide convenience for humans in their lives and related activities. Alongside IoT's potential, security and privacy concerns have been highlighted in the IoT architecture. One weakness of the IoT system is the widespread deployment of sensor nodes with wireless connections. Additionally, the limited resources of these sensor nodes pose a challenge in designing and implementing security solutions for the IoT infrastructure. In this article, we plan to deploy a Network Intrusion Detection System (NIDS) for IoT infrastructure. This system is towards to runs on the Swarm Learning framework, which supports decentralized machine learning models to ensure data distribution during training. This framework also operates on Ethereum – an open-source blockchain platform - to ensure authenticity and security while training decentralized machine learning models. We experiment with various scenarios using the DNN model via the CiCIoT2023 dataset and the CiCIoMT24 dataset. The results demonstrate that our proposed system ensures accuracy comparable to centralized machine learning and Federated Learning models. In addition, we also tested and evaluated based on training time and resource usage, thereby concluding that the cost and effectiveness of the Swarm Learning system is better than Federated Learning.**

***Index Terms*—NIDS, Swarm Learning, Federated Learning, IoT, Security**

I. Introduction

Internet of Things (IoT) is an emerging technology that connects smart devices, allowing them to communicate with each other and provide better services to users. IoT transforms traditional electronic devices, such as sensors, actuators, RFID tags, mobile phones, etc., into smart objects by managing and controlling their underlying technologies. Thus, it enables these objects to see, hear, think, and perform certain tasks by allowing them to synchronize and share information. To improve the quality of life, IoT offers great potential and opportunities in various application areas, such as smart cities, smart buildings, smart agriculture, and healthcare. Besides these smart applications, IoT has some challenges, such as sharing data between components in IoT, facing security issues, and user privacy.

The Intrusion Detection System (IDS) for the Internet of Things (IoT) is a specialized security framework designed to monitor and analyze network traffic across IoT devices. Due to the constrained resources of IoT devices, the IDS is often designed and implemented at the gateway of IoT networks, also known as Edge Devices, so the IDS is called a Network Intrusion Detection System (NIDS). This system is crucial for maintaining the integrity, confidentiality, and availability of IoT networks, ensuring that even the most resource-constrained devices are protected against sophisticated cyber threats.

The development of machine learning, particularly deep learning, in recent years has opened up many opportunities for security solutions in IoT. Integrating machine learning or deep learning into NIDS brings significant benefits in protecting IoT devices and data from attacks. In particular, distributed machine learning architectures ensure that data remains stored on edge devices throughout the entire model training process, contributing to enhanced security for IoT systems as well as improved accuracy of machine learning models. Federated Learning (FL) [1], proposed by Google in 2016, has become a popular framework in decentralized machine learning architecture. However, FL still has some weaknesses, such as i) the existence of a central node and ii) the lack of node authentication. Various proposals have been made to improve FL and address these weaknesses. Additionally, in 2021, HPE introduced a new framework to overcome both of these weaknesses, known as Swarm Learning (SL) [2]. SL places the model aggregator across all member nodes. After each training round, SL selects a node as the leader to receive the weight files, aggregate them, and share the global weight file with other member nodes. Moreover, SL operates on a blockchain platform to ensure the authenticity of the nodes participating in the network.

This paper aims to provide insight into comparing centralized and distributed machine learning architectures for NIDS. In this paper, we delve into the design, implementation, and experimentation of NIDS on three architectures: Centralized

979-8-3315-4246-7/24/$31.00 ©2024 IEEE
DOI 10.1109/ACOMPA64883.2024.00019

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

Machine Learning (CL), FlwrBC (an improvement of FL), and Swarm Learning (SL). We conducted experiments and evaluated these three architectures on two different aspects: the model's accuracy with varying data distribution ratios across the nodes and the performance and training time. The main contributions of this paper are summarized as follows:

- The design and implementation of NIDS on three Machine Learning (ML) architectures: Centralized Machine Learning, FlwrBC, and Swarm Learning;
- Training a Deep Learning model (DNN) on all three architectures using two datasets, CiCIoT2023 and CiCIoMT24, with different configurations;
- Evaluating all three architectures in terms of model accuracy, performance, and system training time.

The remainder of the paper is structured as follows: Section II summarizes related works on Machine Learning-based NIDS; our proposal is described in Section III; Next, Section IV illustrates our experiments and results; The paper ends in Section V with our conclusion and future works.

II. Related works

Many ML integration solutions have been studied to improve the accuracy and performance of intrusion detection systems. A hybrid method combining a stacked non-symmetric deep auto-encoder and support vector machine (SVM) is proposed [3], trained on KDD Cup' 99 dataset. Xuan et al [4] introduced a DNN-based real-time intrusion detection system utilizing a lightweight feature extraction. In the collaborative learning context, numerous research projects are currently being conducted. A LSTM-based FL framework [5] is proposed as an intelligent intrusion detection method that uses FL to detect high-risk malicious behaviors, with its dataset adapted from the open-source SEA dataset. [6] developed a distributed, FL-based system for intrusion detection in IoT scenarios that ensures high accuracy, scalability, and enhanced confidentiality of both data and models through differential privacy techniques.

To enhance trust and transparency in the system, blockchain is integrated with FL-based NIDS research. A Hierarchical Blockchain-based Federated Learning (HBFL) framework is proposed [7] to establish a consortium of organizations that mutually benefit from shared Cyber Threat Intelligence (CTI) while preserving the confidentiality of sensitive training log datasets internally. By integrating blockchain-based auditability, privacy-preserving mechanisms, and encrypted model aggregation with Fully Homomorphic Encryption (FHE) using CKKS, Duy et al. [8] present a federated threat hunting approach for SDN-based IIoT networks.

Additionally, to further decentralization, FlwrBC framework integrates the Flower federated learning framework with the Ethereum blockchain. It addresses the single point of failure (SPOF) challenge by storing global model parameters in a distributed storage system and using a cloud server as the central server and the Inter Planetary File System (IPFS) for distributed file storage to protect against potential attacks. Figure 3 describes the architecture of the FlwrBC framework.

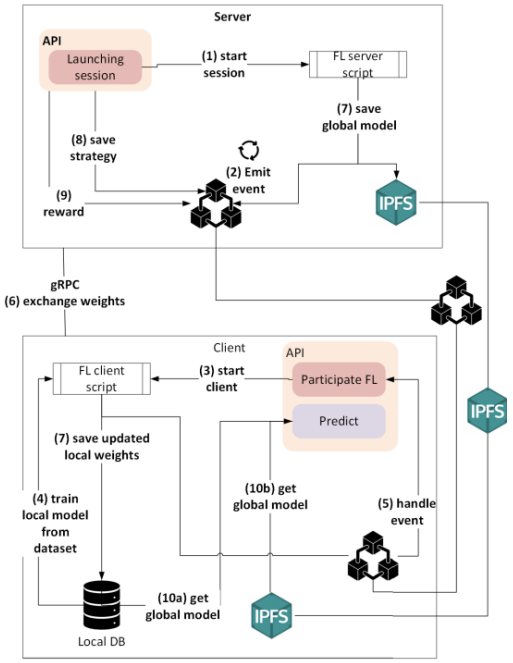


Fig. 1. FlwrBC [9] architecture

While the issue of single point of failure (SPOF) is partially addressed by storing global model parameters in a distributed storage system, the risk of SPOF associated with the central server still persists. This paper has aimed to provide a solution for this problem by using Swarm Learning framework, which allows enable peer-to-peer collaborative learning. Table I summarizes some related works close to our research.

III. Our proposal

In this section, we will present the architecture and operational mechanisms of the Swarm Learning framework used in this article.

A. System design

Shown in Figure 2, the system consists of three clusters, where each cluster has a NIDS installed with SL framework. Each NIDS collects network traffic data from IoT devices in the same cluster, then performs model training and shares learning information with other clusters.

B. System implementation

1) Swarm Learning: SL is divided into separate nodes built using containers. Here are the functions of each container:

- SL node is the core of SL. The SL node operates in coordination with all other SL nodes in the network. It regularly shares its learning with other nodes and combines its insights. SL nodes act as an interface between the application and other SL components. SL nodes take care of securely distributing and merging model weights.
- Swarm Network (SN) nodes combine to form the blockchain network. The current version of SL uses

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE RECENT RESEARCH WORKS FOR NIDS

Paper	SPOF Mitigation	Blockchain	Approach	IDS	Dataset
Emad et al. [3]	No	No	Centralized	Yes	KDD Cup' 99
Realguard [4]	No	No	Centralized	Yes	CIC-IDS2017
Ruijie Zhao et al. [5]	No	No	FL	Yes	SEA
HBFL [7]	No	Yes	FL	Yes	NF-BoT-IoT-v2
Fedchain-Hunter [8]	No	Yes	FL	Yes	InSecLab-IDS-2021 Kitsune Network Attack CICIDS-2017 InSDN
FlwrBC [9]	Medium	Yes	FL	No	CIFAR-10
Our paper	High	Yes	SL	Yes	CiCIoT2023 CiCIoT2024

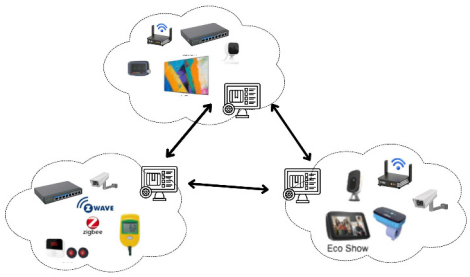


Fig. 2. Overall SL architecture for IoT intrusion detection

the open-source version of Ethereum as the underlying blockchain platform. SN node interact with each other using this blockchain platform to maintain and track progress. SN nodes use this state and progress information to coordinate the activities of other SL components. Users can look up blockchain information and view all activity history. Users have the flexibility to stop SL after completing the training process. When the user reboots the blockchain network, the existing history can be accessed using the *get* or *list* command of the Swarm Learning Command Interface (SWCI) node management interface. (A Sentinel node is a special SN node, responsible for initializing the blockchain network).

- The Swarm Operator (SWOP) node can manage SL operations. The SWOP node is responsible for performing assigned tasks. The SWOP node can only perform one task at a time. The SWOP node helps perform tasks such as starting and stopping SL runs, building and upgrading ML containers.
- The Swarm Learning Command Interface (SWCI) node is the command interface for the SL framework. It is used to monitor SL framework. SWCI nodes can connect to any SN node in the SL network for management. The SWCI node has 2 main tasks to perform: giving orders and monitoring, assigning the task of building the environment and training the model to the SWOP node.
- ML node is the environment where model training is performed, each ML node will be attached to only one node SL and vice versa.

Only metadata is written to the blockchain, the model and data itself are not stored.

The SL framework can convert any machine learning program based on Keras or PyTorch (Python3) into an SL program by making some simple changes to the model training code using the SwarmCallback API. Here the team works with a model based on Keras Tensorflow, the SwarmCallback API will act as a Keras callBack function that acts as a bridge between the ML node and the SL node. SwarmCallback will extract the model weights one at a time and send them to node SL. Node SL shares information with other SL nodes in the network, synthesizes new weights and sends them to SwarmCallback so this function can update the local model.

2) FlwrBC: The architecture of the FlwrBC framework, shown in Figure 4, includes the following components:

- Blockchain: FlwrBC's blockchain and smart contracts are built on the open-source Truffle toolkit (Personal Etherium Blockchain Builder)
- BackEnd: Machine learning models are built based on Python libraries such as Tensorflow, Pytorch, SkLearn. The framework then uses the RPC protocol to perform an exchange of model parameters between the client and the server
- FrontEnd: User communication framework based on web application platform using Web3 FastAPI engine
- IPFS: Global model of framework stored and shared on Pinata's cloud platform

Compared to SL, FlwrBC has only one 'blockchain' that manages the machines participating in the training process instead of each participating machine connecting to a block.

IV. EXPERIMENTAL EVALUATION

In this section, we will present in detail the experimental environment of the previously presented method. At the same

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

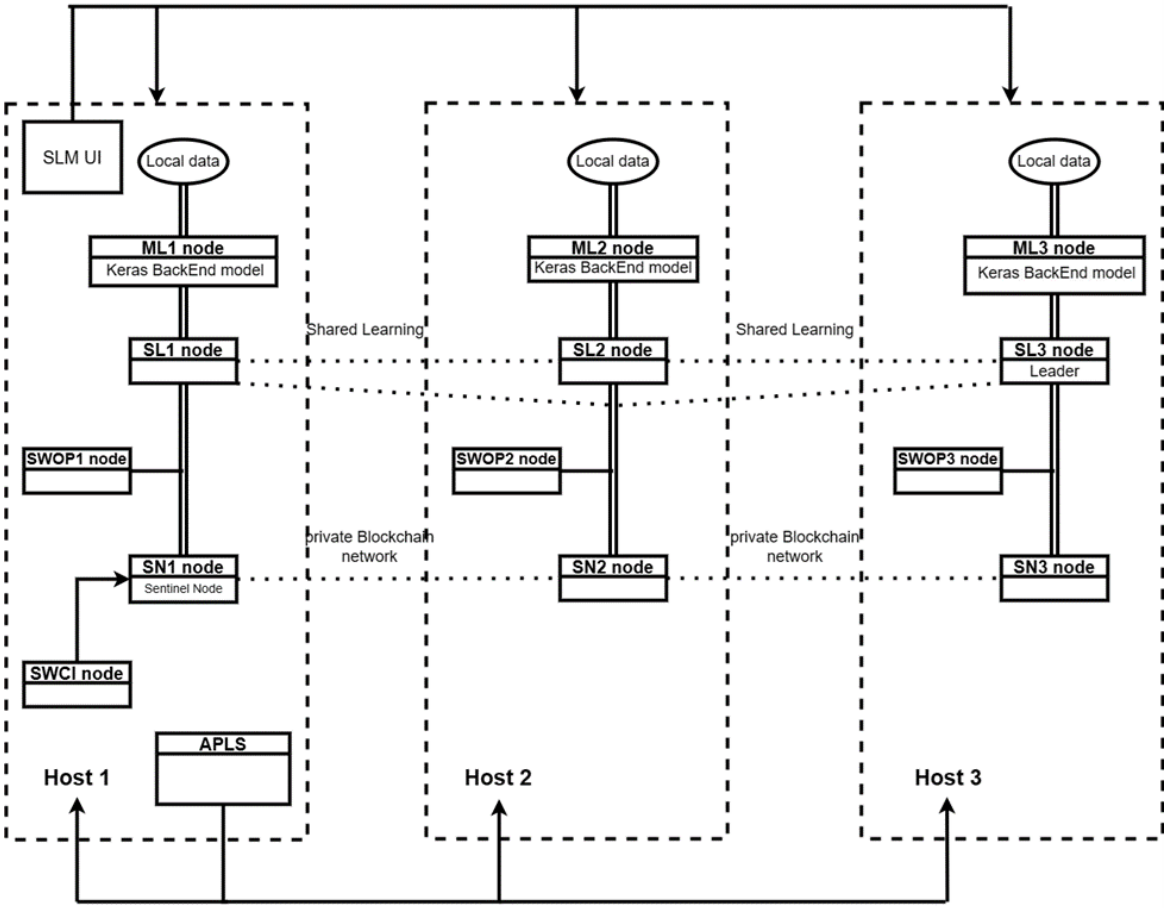


Fig. 3. Swarm Learning architecture [2]

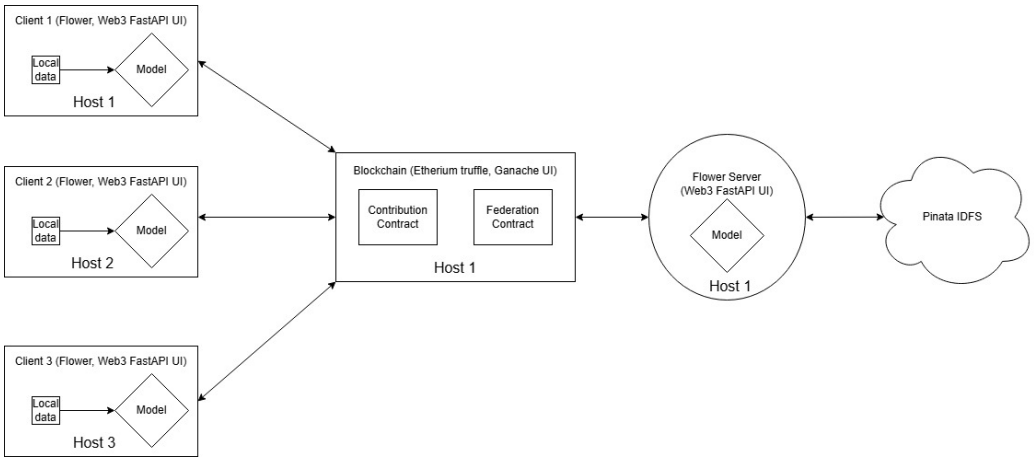


Fig. 4. FlwrBC workflow [9]

time, we build scenarios and discuss the results achieved during the experiment.

A. Datasets description

The performance of our proposed model is assessed using the CiCIoT2023 [10] and CiCIoMT2024 [11]

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

1) CiCIoT2023 [10]: Includes 46 features, 34 labels, we evaluate, group attack labels, and convert labels from letters to numbers as follows: 'BenignTraffic': 0, 'Dictionary Brute Force': 1, 'DDoS': 2, 'Dos': 3, 'Mirai': 4, 'Recon': 5, 'Spoofing': 6, 'Web': 7. The table II shows the distribution of instances for each main category in the CiCIoT2023 dataset.

TABLE II
NUMBER OF INSTANCES FOR EACH CATEGORY IN CICIOT2023 DATASET

Label	Attack	Number of Instances
0	Benign	1,098,195
1	Dictionary Brute Force	13,064
2	DDoS	33,984,560
3	DoS	8,090,738
4	Mirai	2,634,124
5	Recon	354,565
6	Spoofing	486,504
7	Web	24,829

2) CiCIoMT2024 [11]: Includes 44 features, 19 labels, we evaluate, group attack labels, and convert labels from letters to numbers as follows: 'Benign': 0, 'DDoS': 1, 'DoS: 2, 'MQTT': 3, 'Recon': 4, 'ARP Spoofing': 5. The table III shows the distribution of instances for each main category in the CICIoMT2024 dataset.

TABLE III
NUMBER OF INSTANCES FOR EACH ATTACK CATEGORY IN CICIOMT2024 DATASET

Label	Attack Category	Number of Instances
0	Benign	230,339
1	DDoS	5,846,623
2	DoS	2,222,205
3	MQTT	326,653
4	Recon	131,402
5	ARP Spoofing	17,791

B. Evaluation metrics

To evaluate the results recorded after the model training process, we make an assessment based on the criteria described in Table IV:

TABLE IV
EVALUATION METRICS

Metric	Formula / Function / Desciption
Accuracy	(TP + TN) / (TP + FP + TN + FN)
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1-Score	2 x precision × recall / (precision + recall)
Time	Model training time
CPU-Usage	Total CPU usage during training
MEM-Usage	Total Memory usage during training

True Positives (TP) are correctly predicted positive cases, while False Positives (FP) are incorrect positive predictions. True Negatives (TN) are correctly predicted negative cases, and False Negatives (FN) are incorrect negative predictions. In addition to performance metrics like accuracy, precision, recall, and F1-Score, it's crucial to consider computational resources. Time measures how long model training takes, CPU-Usage tracks the CPU resources used, and MEM-Usage indicates the memory consumed. These metrics help ensure the model is both effective and resource-efficient.

C. Experimental settings

Both SL and FlwrBC systems use the same Weight Federated Averaging model consolidation algorithm. The algorithm for calculating the weight of the global model is based on calculating the weighted average of the local models.

The experimental environment includes Python and necessary libraries such as numpy, pandas, tensorflow, dask, matplotlib, and sklearn.

The SL and FL system were deployed on 3 machines with the following configuration: 4 core, 32GB RAM, 100GB DISK. The CL system was deployed on Google Collab and on the third machine :4 core, 32GB RAM, 100GB DISK. In this study, CL, FL, and SL use the same DNN model, shown in table V

TABLE V
TRAINING MODEL CONFIGURATION

Layer	Units	Activation function
Dense	50	ReLU
Dense	25	ReLU
Dense	10	Softmax

The CiCIoT2023 dataset is divided into 2 balanced and imbalanced scenarios in terms of data quantity (1:1:1 and 1:2:3)

The CiCIoMT24 data set is divided into 2 Independent & Identically Distributed (IID) and non-IID scenarios regarding label distribution (scenario 1 1:1:1 and scenario 2 takes the data in scenario 1, deletes 67% of label number 1 data in host 1; 67% of label number 2 data in host 2; 67% of 4 labels number 0, 3, 4, 5 in host 3)

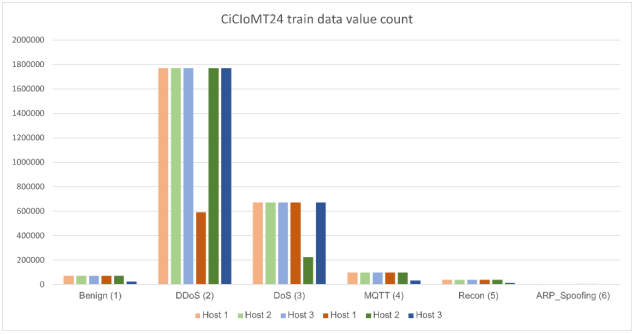


Fig. 5. CiCIoMT24 train data scenario 2

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
EXPERIMENTAL RESULTS ON CICIOT23 DATASET

	Article [10]	CL*	SL case 1	FlwrBC case 1	SL case 2	FlwrBC case 2
Accuracy	99.11%	99.15%	99.09%	99.06%	99.03%	99.06%
Precision	67.94%	94.20%	91.56%	69.18%	94.09%	93.06
Recall	90.66%	71.07%	66.98%	64.25%	65.20%	66.69%
F1-Score	69.72%	75.84%	69.86%	65.70%	67.78%	70.39%
Time		1h	1h32m	2h10m	2h10m	2h52m

* Experimented on Google Colab.

D. Experimental results

1) CiCIoT23: The training results between the two systems SL and FlwrBC are not affected by the difference in the distribution of the number of data samples, as demonstrated in Table VI. The experimental results show high accuracy, with a difference less than 0.1% compared to the article [10] and the results CL. Parameters such as Precision, Recall and F1-Score all give good results, reflecting the model's ability to detect network intrusions for IoTs.

In terms of time, SL has 25-30% faster training time than FlwrBC.

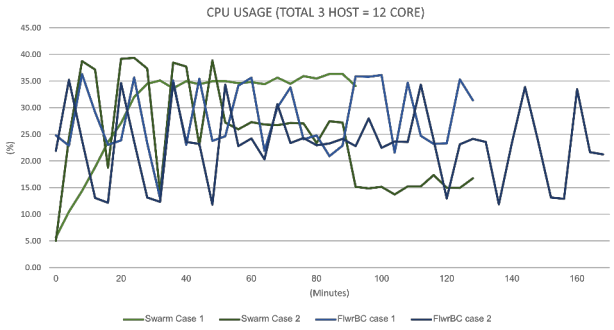


Fig. 6. Graph of CPU usage on CiCIoT23 dataset

Regarding CPU usage, SL reaches its maximum value at 39.38% higher than FlwrBC at 36.32%. The Graph 6 has a variable form because both SL and FlwrBC have a model merging process, with only leader for SL and server for FlwrBC works during merging and then the system consumes less CPU resources than during training.

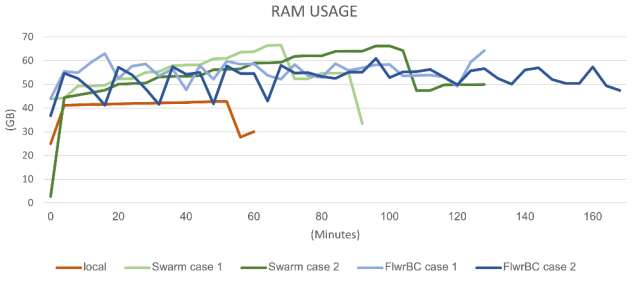


Fig. 7. Graph of RAM usage on CiCIoT23 dataset

Regarding RAM usage, SL uses a maximum of 66GB of ram, higher than FlwrBC's 64GB of ram and CL's 42 GB of ram. This information is shown in Graph 7.

TABLE VII
EXPERIMENTAL RESULTS ON CICIOMT24 DATASET

	Article [11]	CL	SL case 1	FlwrBC case 1	SL case 2	FlwrBC case 2
Accuracy	78.05%	99.61%	99.56%	99.55%	91.16%	89.99%
Precision	76.81%	99.62%	94.59%	95.71%	93.95%	92.78%
Recall	76.02%	99.61%	91.28%	90.21%	84.78%	83.09%
F1-Score	73.35%	99.60%	92.62%	92.15%	87.98%	86.28%
Time		48m20s	42m	1h2m	41m	57m

2) CiCIoMT24: The training results between the two systems, SL and FlwrBC, are described in Table VII. Both of them show good results in the case of complete data. The difference between their results compared to the article and CL training results is less than 0.1%. However, in scenario 2, SL gives 2% better results than FlwrBC and 10% lower results than CL. The difference occurs when scenario 2 deletes data samples in different samples, making the total number of data samples in scenario 2 lower than scenario 1 and lower than the original data.

In terms of time, SL has a training time 28-33% faster than FlwrBC and faster than CL.

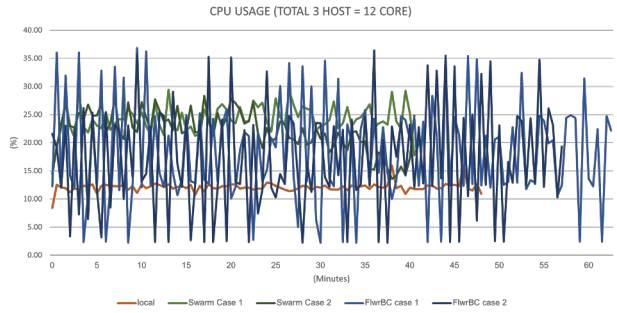


Fig. 8. Graph of CPU usage on CiCIoMT24 dataset

Regarding CPU usage, FlwrBC reaches its maximum value at 36.38%, higher than SL at 29.39% and CL 16.08%. The Graph 8 has a variable form because both SL and FlwrBC have model merging processes, only leader for SL and server for FlwrBC works during merging and then the system consumes less CPU resources than during training.

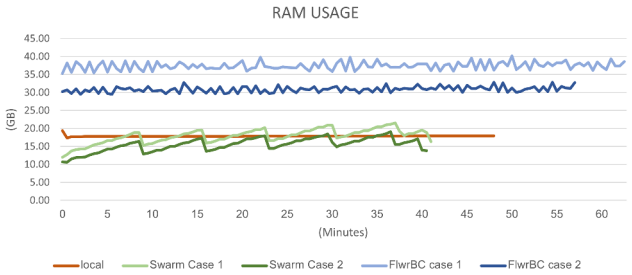


Fig. 9. Graph of RAM usage on CiCIoMT24 dataset

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

In terms of RAM usage, SL is similar to CL and is approximately half the amount of RAM that FlwrBC uses. This information is shown in Graph 9.

E. Analytical

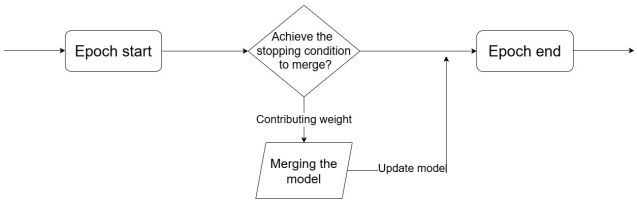


Fig. 10. Stop mechanism of SL

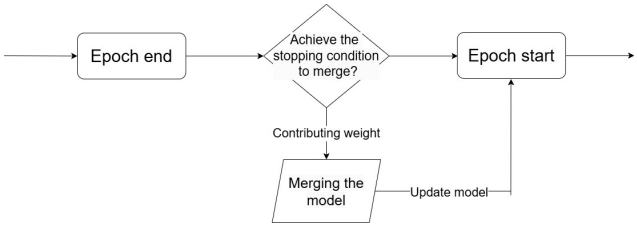


Fig. 11. Stop mechanism of Flower

For FL in general and the Flwr framework in particular, the stopping condition is when the devices complete a fixed number of epochs during local training (Figure 11). However, the SL framework allows flexible configuration of merge and sync timing between devices based on predefined data intervals, without waiting for the epoch to stop (Figure 10. This creates seamlessness in the training process and optimizes training time compared to Flwr.

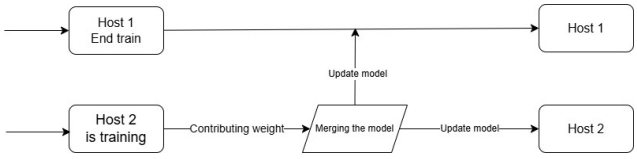


Fig. 12. Weighted contribution mechanism of SL

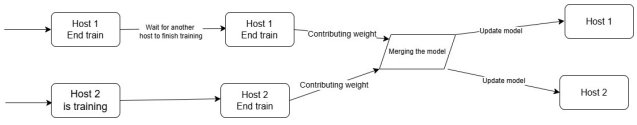


Fig. 13. Weighted contribution mechanism of Flower

Because it is not limited by epoch, hosts of SL (Figure 12 do not need to wait for other hosts to perform local training but can perform model fusion based on the initial customization, resulting in shorter training times. Additionally, hosts with larger amounts of data will have a greater influence on the global model, because once a host completes the training process, it does not participate in contributing weights. This facilitates achieving a higher level of precision. In contrast, in the Flwr system showing in Figure 13, hosts must synchronize the training process before performing the merging process. Additionally, hosts with a smaller amount of data still have the same ability to contribute as hosts with a larger amount of data.

V. Conclusion and Future Works

In this paper, we implemented and evaluated centralized and distributed machine learning models, including FlwrBC and Swarm Learning, for a network intrusion detection system in the Internet of Things infrastructure. We conducted experiments by training a DNN model on centralized and distributed machine learning architectures. The results indicate that FlwrBC and Swarm Learning achieved accuracy comparable to centralized machine learning during training. Regarding time efficiency, Swarm Learning outperformed FlwrBC on the CiCIoT2023 and CiCIoMT2024 datasets. Notably, Swarm Learning had a shorter training time than Centralized Learning and FlwrBC. Additionally, Swarm Learning consumed fewer resources than FlwrBC.

In the future, we have some plans as follows:

- Apply more machine learning models other than DNN to find the most suitable model;
- Optimize the algorithm to improve accuracy as well as model training time to achieve better results;
- Deploy and collect a separate dataset to check the practicality of deployment and the accuracy of the model;
- Optimize SL's leader selection algorithm, bringing more fairness to hosts participating in the training process.

Acknowledgment

This research was supported by The VNUHCM-University of Information Technology's Scientific Research Support Fund.

References

[1] J. Konecn˘ y, H. B. McMahan, F. X. Yu, P. Richt ‘ arik, A. T. Suresh, ´ and D. Bacon, "Federated learning: Strategies for improving communication efficiency," arXiv preprint arXiv:1610.05492, 2016.

[2] Swarm learning: turn your distributed data into competitive edge [White paper]. Hewlett Packard Labs. https://www.labs.hpe.com/pdf/Swarm_Learning.pdf

[3] E.-u.-H. Qazi, M. Imran, N. Haider, M. Shoaib, and I. Razzak, "An intelligent and efficient network intrusion detection system using deep learning," Computers and Electrical Engineering, vol. 99, p. 107764, 2022, doi: 10.1016/j.compeleceng.2022.107764.

[4] X.-H. Nguyen, X.-D. Nguyen, H.-H. Huynh, and K.-H. Le, "Realguard: A lightweight network intrusion detection system for IoT gateways," Sensors, vol. 22, no. 2, p. 432, 2022, doi: 10.3390/s22020432.

[5] R. Zhao, Y. Yin, Y. Shi, and Z. Xue, "Intelligent intrusion detection based on federated learning aided long short-term memory," Physical Communication, vol. 42, p. 101157, 2020, doi: 10.1016/j.phycom.2020.101157.

[6] J. A. de Oliveira, V. P. Gonçalves, R. I. Meneguette, R. T. de Sousa, D. L. Guidoni, J. C. M. Oliveira, and G. P. Rocha Filho, "F-NIDS — A network intrusion detection system based on federated learning," Computer Networks, vol. 236, p. 110010, 2023, doi: 10.1016/j.comnet.2023.110010.

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.

[7] M. Sarhan, W. W. Lo, S. Layeghy, and M. Portmann, “HBFL: A hierarchical blockchain-based federated learning framework for collaborative IoT intrusion detection,” Computers and Electrical Engineering, vol. 103, p. 108379, 2022, doi: 10.1016/j.compeleceng.2022.108379.

[8] P. T. Duy, N. H. Quyen, N. H. Khoa, T.-D. Tran, and V.-H. Pham, “FedChain-Hunter: A reliable and privacy-preserving aggregation for federated threat hunting framework in SDN-based IIoT,” Internet of Things, vol. 24, p. 100966, 2023, doi: 10.1016/j.iot.2023.100966

[9] N. T. Cam and V. T. Kiet, “FlwrBC: Incentive mechanism design for federated learning by using blockchain,” IEEE Access, vol. 11, pp. 107855-107866, 2023, doi: 10.1109/ACCESS.2023.3320045.

[10] Neto, E.C.P.; Dadkhah, S.; Ferreira, R.; Zohourian, A.; Lu, R.; Ghorbani, A.A. CiCIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. Sensors 2023, 23, 5941.

[11] Dadkhah, S., Carlos Pinto Neto, E., Ferreira, R., Chukwuka Molokwu, R., Sadeghi, S., & Ghorbani, A. (2024). CICIoMT2024: Attack Vectors in Healthcare devices-A Multi-Protocol Dataset for Assessing IoMT Device Security. Preprints. https://doi.org/10.20944/preprints202402.0898.v1

Authorized licensed use limited to: King Mongkut's University of Technology Thonburi. Downloaded on October 13,2026 at 10:14:06 UTC from IEEE Xplore. Restrictions apply.